Relazione Finale Progetto PISSIR

Connected Games Platform (GameBar)

Autore: Gueli Alessandro (20038749) & Albitres Farro Jesus Miguel (200038994)

Corso: Progettazione e Implementazione di Sistemi Software in Rete (PISSIR)

Università del Piemonte Orientale (UPO)

01. Specifiche Funzionali

Le specifiche funzionali definiscono i compiti che il sistema software deve assolvere. Il progetto **GameBar** nasce per digitalizzare e connettere i giochi fisici presenti all'interno di locali di intrattenimento (es. bar, sale giochi), consentendo la gestione di partite in tempo reale, tornei e statistiche utente.

Le funzionalità del sistema si dividono in due macro-classi principali:

1. **Controllo Locale (Edge Computing & IoT):**
    - **Rilevamento Eventi:** Interfacciamento con i tavoli da gioco fisici (es. calciobalilla, freccette, ping pong) tramite sensori o nodi edge simulati per rilevare l'inizio partita, l'assegnazione dei punti (es. *Goal*) e la fine della partita.
    - **Funzionamento Offline:** Capacità del nodo periferico di continuare a operare in caso di caduta della connessione di rete, memorizzando temporaneamente gli eventi in un buffer locale per poi sincronizzarli alla riconnessione.
    - **UI Locale / Monitoraggio:** Visualizzazione dello stato dei tavoli e delle partite in corso.
2. **Accessibilità via Internet e Cloud:**
    - **Gestione Centralizzata:** Accesso globale alla piattaforma tramite un'applicazione Web (Frontend) fruibile da rete pubblica.
    - **Autenticazione e Ruoli:** Gestione differenziata degli accessi per Giocatori, Amministratori Locali e Amministratori di Piattaforma tramite token JWT.
    - **Tornei e Classifiche:** Creazione di tornei locali, generazione automatica dei match e aggiornamento dinamico delle statistiche globali degli utenti.

02. Analisi della Tecnologia

La scelta tecnologica per un sistema distribuito che unisce il mondo IoT (dispositivi fisici) al Web richiede l'analisi di vincoli specifici:

- **Installazione in Reti Private:** I tavoli da gioco si trovano solitamente dietro router locali con NAT/Firewall. Non è pensabile configurare port-forwarding manuale su ogni router di ogni locale. Di conseguenza, l'architettura deve basarsi su protocolli con comunicazione *outbound* (in uscita), dove è il dispositivo locale a inizializzare la connessione verso il cloud.

- **Accesso da Rete Pubblica:** Gli utenti devono poter interagire con i dati in tempo reale (visualizzare i punteggi live) senza percepire latenze o differenze tra rete interna ed esterna.
- **Traffico Dati e Real-Time:** Il traffico generato dai sensori è costituito da piccoli messaggi frequenti (eventi di gioco). È necessario un protocollo leggero, a basso overhead di rete, capace di garantire consegne affidabili anche su linee instabili.

03. Scelta dell'Approccio Tecnologico

La scelta architetturale chiave (imposta dai requisiti di interazione IoT e real-time) è l'adozione di un **Message Broker basato sul protocollo MQTT (Eclipse Mosquitto)** affiancato a un'architettura a **Microservizi** esposti tramite un **API Gateway (Nginx)**.

Valutazione rispetto alle alternative:

- **MQTT vs HTTP Polling / REST Tradizionale:**
  - L'approccio HTTP tradizionale (polling continuo del client verso il server) avrebbe sovraccaricato la rete e introdotto latenze inaccettabili per il monitoraggio di un match sportivo in tempo reale.
  - MQTT utilizza il pattern **Publish-Subscribe**. Il sensore (Publisher) invia l'evento al Broker solo quando accade qualcosa (es. un gol), e gli interessati (Match Service, Frontend via WebSocket) ricevono l'aggiornamento istantaneamente. Questo riduce drasticamente l'occupazione di banda e disaccoppia completamente chi produce il dato da chi lo consuma.
- **Tolleranza ai guasti e QoS:**
  - Grazie ai livelli di Quality of Service (QoS 1) di MQTT e all'implementazione di un buffer locale sul nodo edge, il sistema garantisce la persistenza e l'ordine degli eventi anche in caso di blackout temporanei della connessione Internet.

04. Architettura del Software

Il sistema adotta un'architettura distribuita a microservizi containerizzati tramite Docker Compose, garantendo il principio di isolamento dei dati (**Database-per-Service**).

Componenti Principali:

1. **API Gateway (Nginx):** Punto d'ingresso unico per tutte le richieste HTTP REST del client, con routing basato sui path (/api/users, /api/games, /api/matches, /api/tournaments, /api/statistics).
2. **Broker MQTT (Mosquitto):** Gestisce il bus di messaggistica asincrona per i dispositivi IoT ed Edge.
3. **Microservizi Backend (Node.js / Express / Sequelize):**
   - *User Service:* Gestione anagrafica e autenticazione (JWT, bcrypt).
   - *Game Service:* Gestione dei locali fisici e dei tavoli/giochi associati.
   - *Match Service:* Gestione dello stato delle partite e ricezione degli eventi MQTT.

    - *Tournament Service:* Gestione delle competizioni e delle classifiche dei tornei.
    - *Statistics Service:* Calcolo e persistenza delle statistiche globali dei giocatori.
4. **Database Isolati (PostgreSQL):** Ogni microservizio possiede il proprio database logico per garantire l'indipendenza dei dati.

05. Descrizione dell'Implementazione

5.1 Modelli di Dati e Persistenza (ORM Sequelize)

I microservizi utilizzano Sequelize per mappare le entità relazionali su PostgreSQL, sfruttando anche i tipi di dati avanzati di PostgreSQL (JSONB) per memorizzare in modo flessibile configurazioni di sensori, array di partecipanti e punteggi strutturati (es. nelle tabelle Match e Tournament).

5.2 Specifiche delle API REST (OpenAPI / Swagger)

Le interfacce di comunicazione tra Frontend e Backend sono formalizzate tramite standard OpenAPI 3.0.3 (integrabili tramite Swagger). Le operazioni principali comprendono:

- POST /api/users/login (Autenticazione e rilascio Token JWT)
- POST /api/matches (Avvio di una nuova partita sul tavolo)
- POST /api/matches/event (Ricezione evento di gioco / Goal)
- POST /api/matches/end (Chiusura match e attivazione calcolo statistiche)

5.3 Gestione dei Topic MQTT

La comunicazione asincrona dei dispositivi fisici è strutturata sui seguenti topic gerarchici:

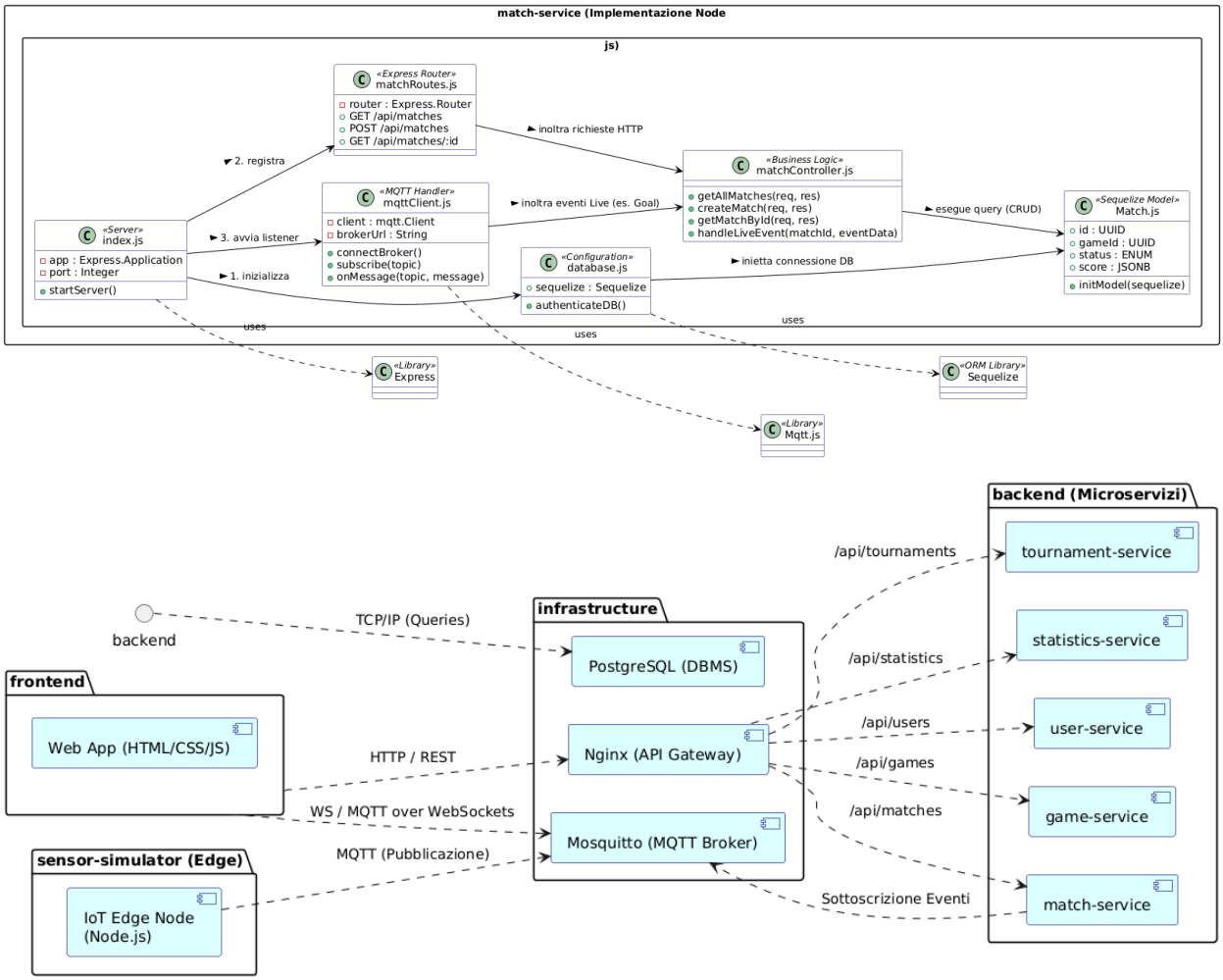
- games/{gameId}/start: Avvio della partita dal tavolo fisico.
- games/{gameId}/goal: Rilevamento punto/gol.
- games/{gameId}/end: Chiusura fisica del match.
- matches/{matchId}/live: Invio dello stato aggiornato ai client connessi in tempo reale.

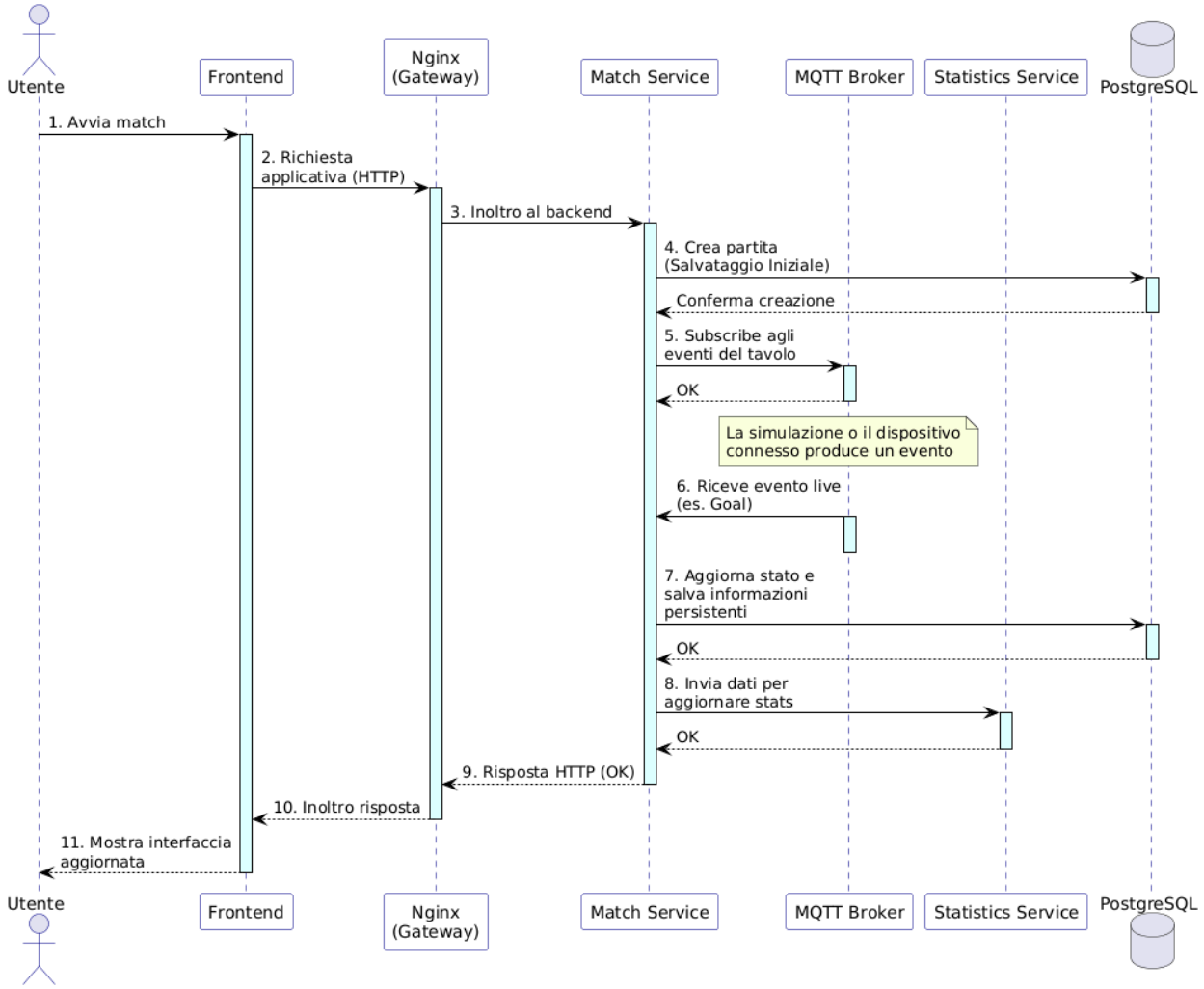
06. Validazione

La fase di validazione del sistema è stata condotta attraverso un processo strutturato di verifica e test:

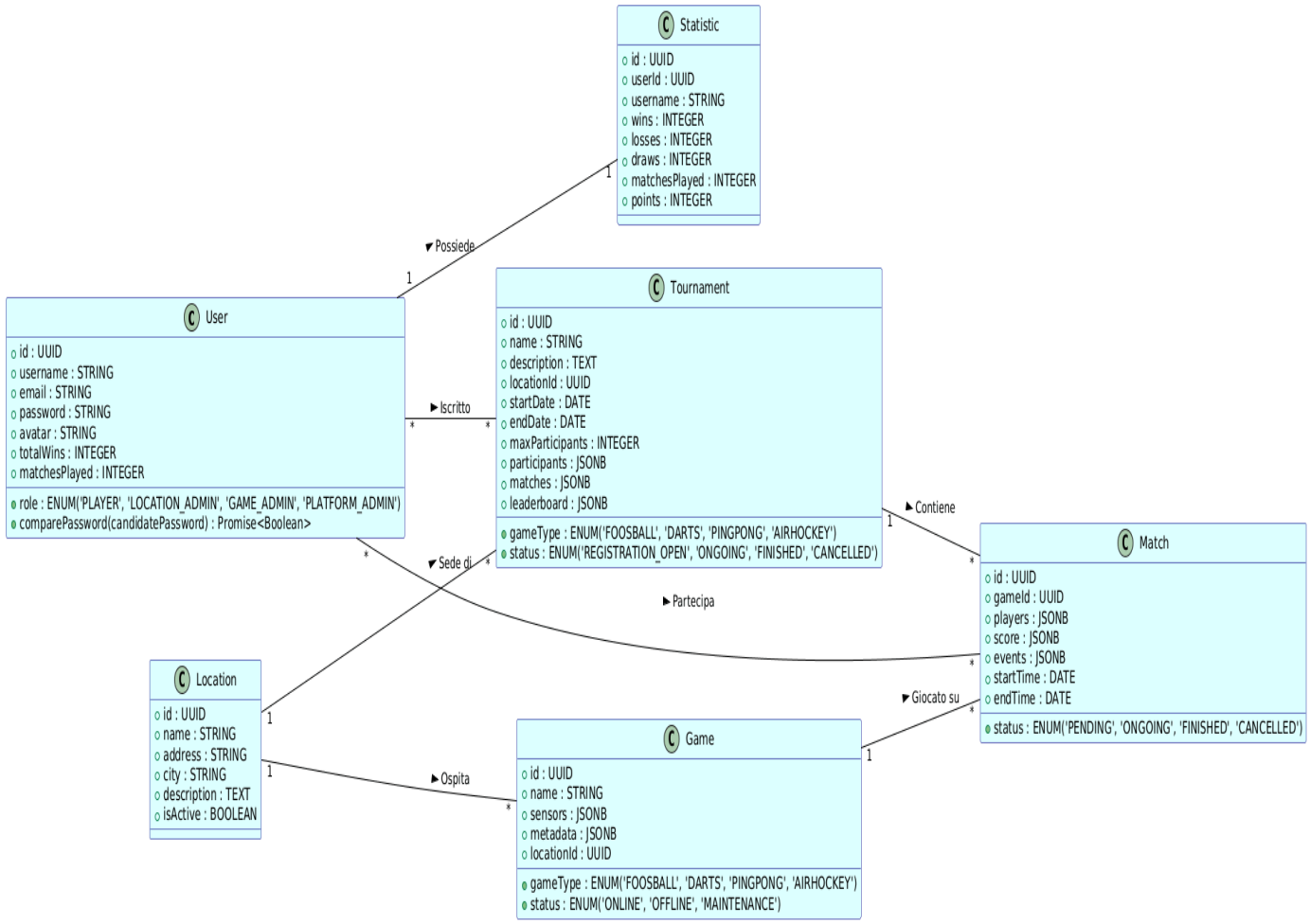
1. **Unit Test e Integrazione:** Verifica dei singoli componenti backend e della correttezza delle query ORM verso i database PostgreSQL isolati.
2. **Test di Comunicazione IoT (Simulatore Edge):** Utilizzo di uno script di simulazione in Node.js per inviare flussi di messaggi MQTT al broker, verificando che il Match Service intercetti correttamente gli eventi e aggiorni i punteggi sul database in tempo reale.
3. **Verifica della Tolleranza ai Guasti (Offline Mode):** Test di disconnessione forzata del broker MQTT durante una partita in corso, riscontrando il corretto accumulo degli eventi nel buffer locale e il successivo reinoltro ordinato alla riconnessione.
4. **Validazione End-to-End:** Controllo del corretto flusso utente dal login via Web App,

passando per l'inoltro tramite Nginx, l'elaborazione del match, fino al ricalcolo finale delle statistiche utente nello Statistics Service.




Utente
Frontend
Nginx (Gateway)
Match Service
MQTT Broker
Statistics Service
PostgreSQL
1. Avvia match
2. Richiesta applicativa (HTTP)
3. Inoltro al backend
4. Crea partita (Salvataggio Iniziale)
Conferma creazione
5. Subscribe agli eventi del tavolo
OK
La simulazione o il dispositivo connesso produce un evento
6. Riceve evento live (es. Goal)
7. Aggiorna stato e salva informazioni persistenti
OK
8. Invia dati per aggiornare stats
OK
9. Risposta HTTP (OK)
10. Inoltro risposta
11. Mostra interfaccia aggiornata
Utente
Frontend
Nginx (Gateway)
Match Service
MQTT Broker
Statistics Service
PostgreSQL


Connected Games Platform (GameBar)
Gestire tutti i Locali
Gestire statistiche globali
Gestire i giochi nel locale
Gestire tornei del locale
Visualizzare statistiche del locale
Autenticazione (Login/Registrazione)
Visualizzare giochi disponibili
Vedere partite giocate
Consultare statistiche personali
Visualizzare classifiche e tornei
Memorizzare dati temporaneamente
«extend»
(se offline)
Inviare dati al Server
Rilevare eventi (Start, Goal, End)
Aggiornare UI Locale
Amministratore Piattaforma
Amministratore Locale
Giocatore
Nodo Edge (Sensori/Simulatore)


Statistic
id : UUID
userId : UUID
username : STRING
wins : INTEGER
losses : INTEGER
draws : INTEGER
matchesPlayed : INTEGER
points : INTEGER
Possiede
1
1
User
id : UUID
username : STRING
email : STRING
password : STRING
avatar : STRING
totalWins : INTEGER
matchesPlayed : INTEGER
role : ENUM('PLAYER', 'LOCATION_ADMIN', 'GAME_ADMIN', 'PLATFORM_ADMIN')
comparePassword(candidatePassword) : Promise<Boolean>
Iscritto
* *
Tournament
id : UUID
name : STRING
description : TEXT
locationId : UUID
startDate : DATE
endDate : DATE
maxParticipants : INTEGER
participants : JSONB
matches : JSONB
leaderboard : JSONB
gameType : ENUM('FOOSBALL', 'DARTS', 'PINGPONG', 'AIRHOCKEY')
status : ENUM('REGISTRATION_OPEN', 'ONGOING', 'FINISHED', 'CANCELLED')
Contiene
1
*
Match
id : UUID
gameId : UUID
players : JSONB
score : JSONB
events : JSONB
startTime : DATE
endTime : DATE
status : ENUM('PENDING', 'ONGOING', 'FINISHED', 'CANCELLED')
Sede di
* *
Partecipa
* *
Location
id : UUID
name : STRING
address : STRING
city : STRING
description : TEXT
isActive : BOOLEAN
1
1
Ospita
*
Game
id : UUID
name : STRING
sensors : JSONB
metadata : JSONB
locationId : UUID
gameType : ENUM('FOOSBALL', 'DARTS', 'PINGPONG', 'AIRHOCKEY')
status : ENUM('ONLINE', 'OFFLINE', 'MAINTENANCE')
Giocato su
1
*